birdclef-2026

12th place solution - MixMax Consistency Regularization

Rank	12
Team	Bobbing Redstart
Author	Antoine Masq
Topic ID	704404
Version	Original

First of all, I'd like to thank the organizers for making the BirdCLEF competitions such a fantastic opportunity to experiment and learn. A lot of solutions this year have been using Perch in one way or another but my first experiments with it weren't promising, so I decided to explore other paths. While I didn't distill Perch's embeddings, their paper sparked the idea for my mixup regularization (it's a kind of knowledge distillation, I guess haha).
Also, I was not expecting to get to the gold medal zone this year, and was even more surprised to not be shaken up out of the top of the leaderboard at the end.

Data

Preprocessing

Empty recordings

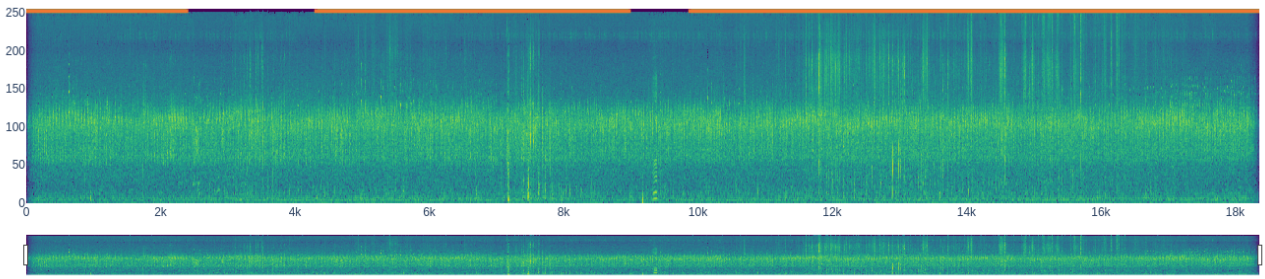
As I previously mentioned in a discussion post, there were corrupted recordings that were just 1s glitchy noises. I tried to download them from the url available in the train.csv file and they appeared to be clean and normal recordings. In total there were 369 files that I could recover that way.

Duplicates

Also I applied torch's hash_tensor function on the recordings and noticed that some files were duplicated. To avoid data leaking I dropped all the hash duplicates. In total there were 86 unique recordings appeared up to three times.

Manual cleaning

By using my visualization tool (that you can find here), I noticed that some minority classes had long recordings where their actual calls were quite sparse, especially for Bos Taurus for which most of the recordings had the same unlabeled background species. This could lead to potential confusion between the unlabeled background species with the target species.
For example:



To avoid such a confusion, I manually annotated time windows where the target species were not calling and removed them from the recordings.

Silent soundscapes

As we only have positive training data, I was a bit worried that my models would become biased toward predicting positive labels. To have some training data where there are no positive labels, I iteratively ran my early models on the soundscapes and flagged those whose maximum values fell below a certain threshold, then manually checked that they were indeed silent, and used them as negative examples to train a new model, repeating the process twice.

Additional data

As noted by @patrob and @kdmitrie in a discussion on the forum, some species had a few examples from Xeno-Canto because they had inactive taxa that were different from iNat. Specifically they flagged that the Hydropsalis maculicaudus had a synonym taxon Antiurus maculicaudus for which there was data on Xeno-Canto. Then I checked for other possible synonyms for minority classes and found a synonym for Rufirallus schomburgkii (Micropygia schomburgkii). In total there were 71 and 92 recordings from Xeno-Canto that could be used for those species.

Pre-training data (Xeno-Canto)

The models were also using backbones that I pre-trained on Xeno-Canto data. You can find the metadata of the files used during the pre-training [here] and I'll also try to include the audio and share the pre-trained weights later on so it can be used by everyone in the next BirdCLEF editions.

Loss

The loss is a custom Asymmetric Loss (ASL) where I used different gammas for each class, that I refer to as CASL (Class-wise ASL). The intuition behind it is to softly down-weight classes that are more likely to have noisy labels. For example a lot of recordings containing unlabeled insect noises or species with short calls might have their calls cut off when randomly sampling 5s windows.

	Aves	Amphibia	Insecta	Mammalia	Reptilia
$$\gamma^+$$	0.5	0	0	0.5	0
$$\gamma^-$$	0.5	0.5	2	0.1	0.5

Another detail is that I reduced the weight of the insect sonotypes' loss for focal recordings since those labels were specific to the soundscapes.

Model

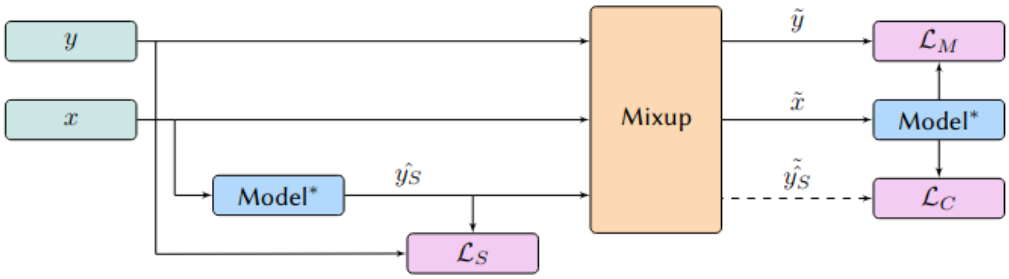
The model itself is a simple EfficientNetv2 backbone (b0, b3 and s) with a 2-layers classification head.

Training

The models were trained with an AdamW optimizer with cosine-annealing scheduling with a learning rate decaying from 5e-4 to 1e-6 with a batch size of 32 samples.

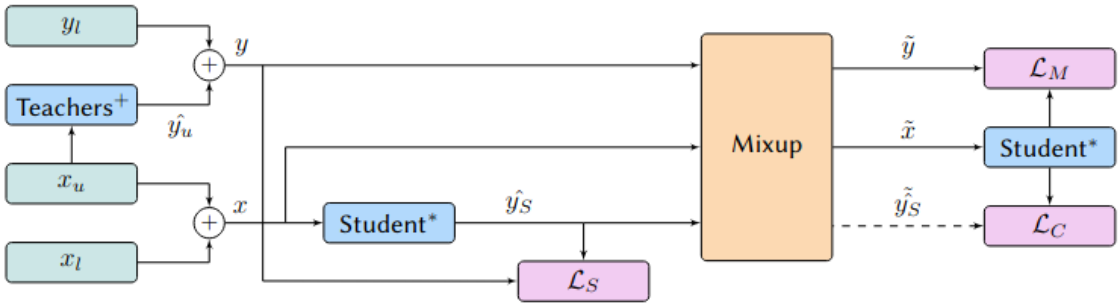
Baseline

The baseline training pipeline was mainly inspired by the Mixmatch paper, and as I said in the introduction, the Perch paper sparked the idea of using mixup consistency as a regularizer. In the paper, they used a mixup that takes the maximum of the labels rather than a linear interpolation, the idea being that if species are present in a recording, they should be present in the mixture regardless of their weight in the interpolation.
I took this a step further, using it as a consistency regularizer to handle noisy labels. The underlying intuition being that if the model detects species in the individual recordings, it should detect it in the mixture as well. Furthermore, if a recording has noisy labels, the model will not be penalized if it predicts the missing species both for the individual recording and the mixture. This acts as a regularizer **and** a self-supervised signal.
Also, in the diagram the "model*" includes the spectrogram transformation and the augmentation module, this means that the model needs to be consistent both between the individual recordings and their mixture, and across different augmentations.
As we take the maximum of the labels in the mix for the regularization, let's give it the fancy name of MixMax consistency regularization 😄



Semi-supervised learning

For the pseudo-labeling part, I simply extended my baseline pipeline by using online pseudo-labeling as it yields more diverse samples from the unlabeled soundscapes. The teachers were the models trained with the baseline pipeline for each fold, their predictions were sharpened and then averaged.



Baseline results

From the results I got, it seems that the CASL and my MixMax regularization were not that strong on their own but seemed to have good synergy. Hopefully, I'll have some time to dive deeper into that for the Working Note.

Also, the effect of pre-training was not negligible and gave a boost of around 0.01 on both private and public LB.

Loss	XC Pre-training	$$\mathcal{L}_C$$	Local Soundscape AUC	Public LB	Private LB
BCE			0.889	0.917	0.917
BCE		☒	0.896	0.917	0.927
BCE	☒		0.900	0.926	0.931
BCE	☒	☒	0.894	0.924	0.932
CASL			0.882	0.913	0.922
CASL		☒	0.901	0.928	0.927
CASL	☒		0.898	0.925	0.931
CASL	☒	☒	**0.918**	**0.938**	**0.940**

Post-processing

The post-processing pipeline included:

- Logits sharpening (T=0.8) before ensembling
- A 2.5s sliding window TTA where the predictions for a window were the maximum of its own and its neighbors'
- Smoothing convolution for Amphibians and Insects
- File-level smoothing (maximum probability across the file)
- Rough priors computed with the same models (simple average for site, hours and months)

Post-processing step	Public LB	Private LB	$$\Delta \text{ Private LB} $$
Ensemble raw predictions	0.9476	0.9413	0
+ logits sharpening	0.9477	0.9418	0.0005
+ sliding window	0.9555	0.9517	**0.0099**
+ smoothing	0.9588	**0.9560**	0.0043
+ priors	**0.9595**	0.9558	-0.0002

Final ensemble

Backbone	Augmentation regime	Mixup$$\alpha$$	Fold	Public LB	Private LB
tf_efficientnetv2_b0	extreme	1	NA	0.947	0.950
tf_efficientnetv2_b3	light	2	NA	0.947	0.950
tf_efficientnetv2_s	strong	0.5	NA	0.941	0.951
tf_efficientnetv2_b0	strong	0.2	NA	0.939	0.941
tf_efficientnetv2_s	strong	0.5	4	0.944	**0.951**
tf_efficientnetv2_b3	light	0.5	0	0.942	0.942
tf_efficientnetv2_b0	strong	0.5	3	**0.949**	0.936
tf_efficientnetv2_s	light	2	1	0.944	0.951
Ensemble				**0.960**	**0.956**

That's it from my side! Again, this was a great opportunity to test, to fail and most importantly: learn!
You can find the source code and the compiled models below, and if you have any question, I'd be happy to answer 😀